Bridge Pattern Assignment

Extending the device and remote example

Design Patterns course

14 November 2026

---

1. Introduction

The Bridge pattern is a structural pattern that separates an abstraction from its implementation. The abstraction keeps a reference to an implementation object and calls its methods instead of implementing everything itself. This way the abstraction and the implementation can be changed separately.

I chose the Java example from Refactoring Guru: https://refactoring.guru/design-patterns/bridge/java/example.

In that example the abstraction is the remote control. `Remote` is an interface, `BasicRemote` implements it, and `AdvancedRemote` extends `BasicRemote` with a `mute()` method. The implementation side is the `Device` interface, which has two implementations: `Radio` and `Tv`. `BasicRemote` has a `Device` field, so the same remote code can be used with different devices. That field is the bridge between the two class hierarchies.

Bridge role	Classes in the example
Abstraction	`Remote`, `BasicRemote`
Refined abstraction	`AdvancedRemote`
Implementor	`Device`
Concrete implementors	`Radio`, `Tv`

The original example shows that a basic remote and an advanced remote can both be used with a radio and a TV. I used this example as the base and added one new device and one new remote to it.

2. New Functionality

I added the following classes:

- `SmartDevice` is an interface that extends `Device`. It adds `browseInternet(String url)` and `getCurrentPage()`.
- `SmartTV` implements `SmartDevice`. It stores the current web page and only allows browsing when the TV is on.
- `SmartRemote` extends `AdvancedRemote`. It adds a `voiceControl(String command)` method.

I added browsing to a separate `SmartDevice` interface instead of adding it directly to `Device`. A radio cannot browse the internet, so it should not have to implement a browsing method. This way the old `Radio` and `Tv` classes do not need any changes, and only smart devices have the browsing methods.

Voice control is a feature of the remote, so I put it in `SmartRemote`. The remote can still be connected to any `Device`. If the device is not a `SmartDevice`, the `open` command prints a message instead of trying to browse.

The voice control supports these commands:

Command	What it does
`power on` / `power off`	Switches the device on or off.
`power` / `power toggle`	Toggles the power state.
`volume up` / `volume down`	Changes the volume by 10.
`volume N`	Sets the volume to `N`. The device limits it to 0–100.
`mute`	Sets the volume to 0.
`channel up` / `next channel`	Changes to the next channel.
`channel down` / `previous channel`	Changes to the previous channel.
`channel N`	Sets the channel to `N`.
`open URL`	Opens the URL on a smart device. On other devices it prints a message.

3. Implementation

I added the new classes to the existing `bridge.devices` and `bridge.remotes` packages. The project structure is:

```
src/bridge/
  Main.java
  devices/
    Device.java
    Radio.java
    Tv.java
    SmartDevice.java
    SmartTV.java
  remotes/
    Remote.java
    BasicRemote.java
    AdvancedRemote.java
    SmartRemote.java
```

3.1 SmartDevice

`SmartDevice` extends `Device` and adds the two browsing methods. The old code still works because it only needs the `Device` methods.

```
package bridge.devices;

public interface SmartDevice extends Device {

    void browseInternet(String url);

    String getCurrentPage();
}
```

3.2 SmartTV

`SmartTV` implements all the methods from `Device` and `SmartDevice`. The volume is limited to 0–100 in the same way as in the original `Radio` and `Tv` classes. The browsing method checks that the TV is on and that the URL is not empty.

```
package bridge.devices;

public class SmartTV implements SmartDevice {

    private boolean on = false;
    private int volume = 30;
    private int channel = 1;
    private String currentPage = "home";

    @Override
    public boolean isEnabled() {
        return on;
    }

    @Override
    public void enable() {
        on = true;
    }

    @Override
    public void disable() {
        on = false;
    }

    @Override
    public int getVolume() {
        return volume;
    }

    @Override
    public void setVolume(int volume) {
        this.volume = Math.max(0, Math.min(100, volume));
    }

    @Override
    public int getChannel() {
        return channel;
    }

    @Override
    public void setChannel(int channel) {
        this.channel = channel;
    }

    @Override
    public void browseInternet(String url) {
        if (!on) {
            throw new IllegalStateException("SmartTV must be on to browse the internet");
        }
        if (url == null || url.isBlank()) {
            throw new IllegalArgumentException("URL must not be empty");
        }
        currentPage = url;
    }

    @Override
    public String getCurrentPage() {
        return currentPage;
    }

    @Override
    public void printStatus() {
```

```
        System.out.println("-----------------------------------");
        System.out.println("| I'm smart TV.");
        System.out.println("| I'm " + (on ? "enabled" : "disabled"));
        System.out.println("| Current volume is " + volume + "%");
        System.out.println("| Current channel is " + channel);
        System.out.println("| Current page is " + currentPage);
        System.out.println("-----------------------------------\n");
    }
}
```

3.3 SmartRemote

`SmartRemote` extends `AdvancedRemote`, so it already has `power()`, the volume and channel methods, and `mute()`. The `voiceControl` method converts the command to lower case and then chooses what to do. The `open` method checks with `instanceof` whether the connected device is a `SmartDevice`.

```
package bridge.remotes;

import bridge.devices.Device;
import bridge.devices.SmartDevice;

import java.util.Locale;

public class SmartRemote extends AdvancedRemote {

    public SmartRemote(Device device) {
        super(device);
    }

    public void voiceControl(String command) {
        if (command == null) {
            throw new IllegalArgumentException("Voice command must not be null");
        }

        String normalized = command.trim().toLowerCase(Locale.ROOT);
        System.out.println("SmartRemote: \"" + command.trim() + "\"");

        if (normalized.isEmpty()) {
            System.out.println("SmartRemote: empty command");
            return;
        }

        switch (normalized) {
            case "power", "power toggle" -> power();
            case "power on" -> device.enable();
            case "power off" -> device.disable();
            case "volume up" -> volumeUp();
            case "volume down" -> volumeDown();
            case "mute" -> mute();
            case "channel up", "next channel" -> channelUp();
            case "channel down", "previous channel" -> channelDown();
            default -> handleParameterizedCommand(normalized);
        }
    }

    private void handleParameterizedCommand(String command) {
        if (command.startsWith("volume ")) {
            Integer volume = parseNumber(command.substring("volume ".length()).trim());
            if (volume != null) {
                device.setVolume(volume);
            }
```

```
            return;
        }

        if (command.startsWith("channel ")) {
            Integer channel = parseNumber(command.substring("channel ".length()).trim());
            if (channel != null) {
                device.setChannel(channel);
            }
            return;
        }

        if (command.startsWith("open ")) {
            open(command.substring("open ".length()).trim());
            return;
        }

        System.out.println("SmartRemote: command not recognized");
    }

    private void open(String target) {
        if (!(device instanceof SmartDevice smartDevice)) {
            System.out.println("SmartRemote: " + device.getClass().getSimpleName()
                    + " does not support internet browsing");
            return;
        }

        if (!device.isEnabled()) {
            System.out.println("SmartRemote: turn the device on before browsing");
            return;
        }

        smartDevice.browseInternet(target);
        System.out.println("SmartRemote: opened " + target);
    }

    private Integer parseNumber(String value) {
        try {
            return Integer.parseInt(value);
        } catch (NumberFormatException e) {
            System.out.println("SmartRemote: '" + value + "' is not a number");
            return null;
        }
    }
}
```

3.4 Demo

`Main` tests four combinations: a basic remote with a TV, an advanced remote with a radio, a smart remote with a smart TV, and a smart remote with a normal TV. The last case checks that the smart remote still works with a device that does not support browsing.

```
public static void main(String[] args) {
    testBasicRemote(new Tv());
    testAdvancedRemote(new Radio());
    testSmartRemote(new SmartTV());
    testSmartRemoteWithClassicDevice(new Tv());
}
```

4. Verification

I compiled the project with JDK 21 and ran the demo:

```
javac --release 21 -encoding UTF-8 -d build (all .java files)
java -cp build bridge.Main
```

I also wrote a separate test class with simple check methods. It tests the following things:

- `SmartTV` is a `Device` and a `SmartDevice`.
- The initial state is off, volume 30, channel 1 and page `home`.
- Browsing while the TV is off throws an `IllegalStateException`.
- Null and blank URLs throw an `IllegalArgumentException`.
- Volume is clamped between 0 and 100.
- Voice commands change power, volume and channel correctly.
- `mute` sets the volume to 0.
- `open` only changes the page when the device is on and is a `SmartDevice`.
- A `SmartRemote` connected to a `Radio` or a normal `Tv` still controls power, volume and channels, and prints a message for `open`.
- Unknown commands and non-numeric values do not change the state.
- The old `BasicRemote` and `AdvancedRemote` still work with the old devices and with `SmartTV`.

The test output was:

```
All 39 bridge checks passed.
```

Here is a part of the output from `bridge.Main`:

```
== Smart remote with a smart TV ==
SmartRemote: "power on"
SmartRemote: "open netflix.com"
SmartRemote: opened netflix.com
SmartRemote: "volume 45"
SmartRemote: "next channel"
Remote: channel up
SmartRemote: "mute"
Remote: mute
-----------------------------------
| I'm smart TV.
| I'm enabled
| Current volume is 0%
| Current channel is 2
| Current page is netflix.com
-----------------------------------

SmartRemote: "open youtube.com"
SmartRemote: opened youtube.com
SmartRemote: "volume up"
Remote: volume up
-----------------------------------
| I'm smart TV.
| I'm enabled
| Current volume is 10%
| Current channel is 2
```

```
| Current page is youtube.com
-----------------------------------

== Smart remote with a regular TV ==
SmartRemote: "power on"
SmartRemote: "volume 70"
SmartRemote: "open example.com"
SmartRemote: Tv does not support internet browsing
-----------------------------------
| I'm TV set.
| I'm enabled
| Current volume is 70%
| Current channel is 1
-----------------------------------
```

The output shows that the smart remote works with the smart TV and that the browsing command is ignored with a normal TV. The old remotes also still print the same status as before.

5. Conclusion

I extended the Refactoring Guru bridge example with a smart TV and a smart remote. The new device supports internet browsing and the new remote supports voice commands. The original classes were not modified.

The main design decision was to put the browsing methods in a new `SmartDevice` interface instead of adding them to `Device`. This keeps `Radio` and `Tv` unchanged and makes it clear which devices support browsing. Another decision was to make `SmartRemote` extend `AdvancedRemote`, so I could reuse the power, volume, channel and mute methods instead of writing them again.

If the connected device is not a smart device, the remote prints a message instead of throwing an exception. This is not required by the assignment, but it makes the demo easier to use and shows the difference between the device types.

I also considered adding `browseInternet` directly to `Device` with a default method. That would have been shorter, but then all devices would appear to support browsing. I also considered making `SmartRemote` a separate class that does not extend `AdvancedRemote`, but that would have duplicated the basic remote methods. For the voice commands, a separate command class for each command would be better if there were many commands, but for this small set a switch is enough.

The project compiled without errors and all 39 tests passed, so the new functionality works as expected.